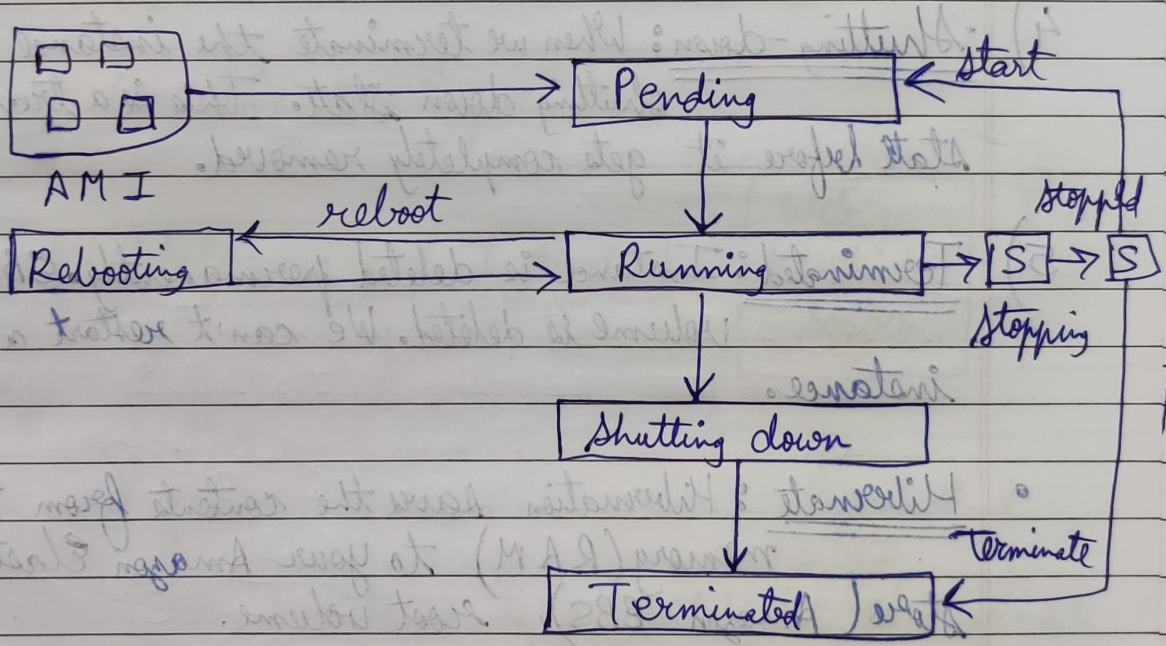
SSH → Secure Shell
RDP → Remote Desktop Protocol

||ॐ नमो भगवते वासुदेवाय नमः||

20 August 2025 — Wednesday — 5:25 AM

* Amazon EC2 Lifecycle — Elastic Cloud Compute

- An EC2 instance is like a virtual server running on AWS infrastructure. Every EC2 instance goes through states (lifecycle) and AWS lets you manage these states depending on your need.



1) Pending: When we first launch an instance, it goes into the pending state. AWS provides resources like CPU, memory, networking, EBS volumes. (Elastic Block Store)

2) Running: Once initialization is complete, the instance is in running state. We can connect via SSH/RDP. Applications & OS are active. We are billed for compute charges in this state.

3) Stopping / Stopped:

- Stopping: When we ~~shut down~~ stop an instance, it shuts down gracefully.
- Stopped: Instance is powered off, but the attached EBS volumes remain intact.
- We can restart later & it will retain data/settings.

4) Shutting-down: When we terminate the instance, it enters shutting down state. This is a transitional state before it gets completely removed.

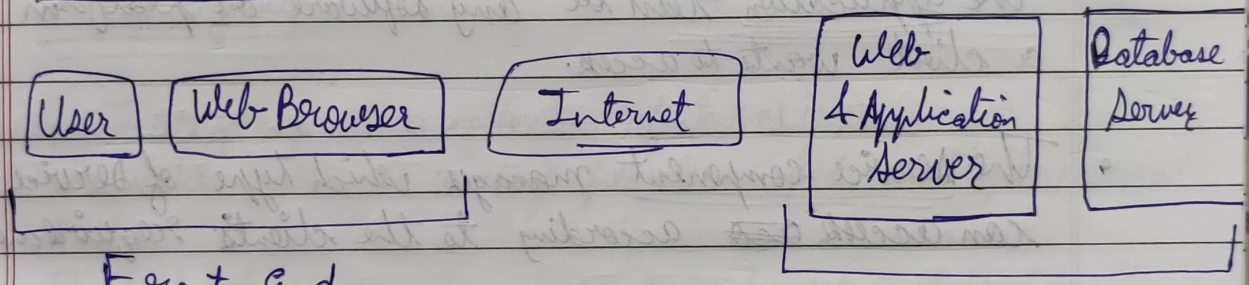
5) Terminated: Instance is deleted permanently. EBS root volume is deleted. We can't restart a terminated instance.

- Hibernate: Hibernation saves the contents from the instance memory (RAM) to your Amazon Elastic Block Store (Amazon EBS) root volume.
- Reboot: An instance reboot is equivalent to an operating system reboot. Data persists.
- S3 (Simple Storage Service):- It is an object service storage service provided by AWS. Objects storage means data is stored as objects, not files or blocks. Each object consists of data + metadata + unique identifier (key).

* **Buckets in S3** : A bucket is a container for storing objects in S3. Each bucket has a unique name globally (across all AWS users). Data inside buckets is organized as a flat structure

* **Objects**: Objects are the actual data files stored inside buckets. Each object consists of data, key (unique identifier) and metadata. A bucket is used to store objects. When data is added to a bucket, Amazon S3 creates a unique version ID & allocates it to the object.

* **Cloud Computing Architecture**



- Cloud Computing Architecture is a combination of components required for a cloud computing service. It ~~consist~~ consists of several components like a front end platform, backend platform, servers, network, Internet service & cloud based delivery service.

- **Front End**: The client uses the front end, which contains a client side interface and application. Both of these components are important to access the cloud computing platform. The front end includes web servers (Chrome, Firefox, Opera, etc.), clients & mobile devices.

- **Back End:** The backend part helps you manage all the resources needed to provide cloud computing service. This cloud architecture part includes a security mechanism, a large amount of data storage, servers, virtual machines, traffic control mechanisms, etc.

* IMP components of Cloud Computing Architecture

- Client Infrastructure is a front-end component that provides a GUI. It helps users to interact with the cloud.

- The application can be any software or platform which a client wants to access.

- The service component manages which type of service you can access according to the client's requirements.

  1) Software as a Service (SaaS)
  2) Platform as a Service (PaaS)
  3) Infrastructure as a Service (IaaS)

- Runtime cloud offers the execution & runtime environment to the virtual machines.

- Storage provides a large amount of storage capacity in the cloud to store & manage data.

- Security in the backend refers to implementing different security mechanisms for secure cloud systems, resources, files & infrastructure to the user.

* IAM Users

- Entity that represents the person or service that uses it to interact with AWS. It consists of a name & unique set of credentials :-
  - Console password
  - Access key
  - MFA device
- Each IAM user is associated with one & only one AWS account does not require a separate payment method.

* IAM Group

- An IAM group is a collection of IAM users.
- It makes easier to manage permissions for multiple users at once.
- Instead of assigning permission to each user individually, you assign them to the group & all users in that group inherit the permissions.

* IAM Role

- It is an identity with specific permissions that can be assumed by trusted entities.
- Unlike a user, a role does not have username & password. Instead it provides, temporary ~~service~~ security credentials.
- Used when AWS services or external accounts need permissions.

* EC2 Instance Metadata

- Metadata is data about the instance itself. It provides information about running instance such as
  - Instance ID
  - Public & Private IP addresses
  - Security groups attached
  - AMI ID (Amazon Machine Image)
  - Availability Zone & Region
  - Instance Type

* EC2 User Data

- User Data is data you pass to an EC2 instance at the time of launch to automate boot tasks. It is executed once during the first boot.
- Mostly used for installing software, running updates, configuring applications.

classmate
Date 7
Page

॥ ॐ नमो भगवते वासुदेवाय नमः ॥

5 October 2025 — Sunday

Q1) Importance of cloud in AI

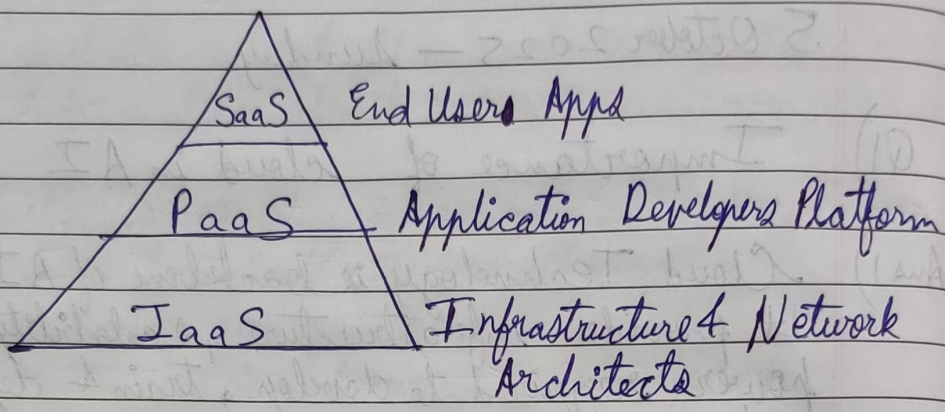
Ans 1) Cloud Technology is backbone of AI today. It provides infrastructure, scalability & computational power required to develop, train & deploy AI models efficiently.

- Scalable Computing Power :- AI models need high processing capabilities, easily provided by cloud by GPUs & TPUs, HPC clusters.
- Data Storage & Access :- Cloud providers virtually infinite storage with object storage services like AWS S3. Data can be centralized, shared & accessed globally by AI systems i.e. no local storage limits.
- AI Services on Demand : Cloud platforms offer ready-to-use AI/ML services which even a small business or students can access enterprise-grade AI without building from scratch.
- Cloud allows continuous Integration & Deployement CI/CD of AI models. We can train a model, test it with real world data, deploy it globally within minutes using cloud endpoints.

This agility makes AI practical in business, not just research.

Q2) Cloud Service Model Diagram

Ans 2)



- Software as a Service: Offers application to customers or organizations through a web browser. Data for the app runs on a server on the network, not through an app on the users computer. Software is sold via subscription.
- Platform as a Service:

1) Software as a Service (SaaS)

- It provides ready-to-use software delivered over internet
- It is used by end users
- Examples are Gmail, Google Drive, MS Office 365
- We just log in & use the app — no coding, no infra setup.

2) Platform as a Service (PaaS)

- It provides a development environment like runtime, frameworks, tools on top of IaaS.
- It is used by developers building & deploying apps.
- Examples are Google App Engine, AWS Elastic Beanstalk, Microsoft Azure App services.
- You focus on coding, platform handles infra + runtime.

3) Infrastructure as a Service (IaaS)

- It provides virtualized hardware resources like Servers, storage & networking.
- It is used by system admins & IT teams.
- Examples are AWS EC2, Google Compute Engine
- You rent infrastructure. instead of buying servers

Q3) Steps to create a user in user group to access the Lambda, S3 & API Gateway services.

Ans3) Steps are :-

1) Login to AWS Management Console. Go to the console → Search for IAM (Identity & Access Management)

2) In IAM, select User groups → Create group give it a name eg Dev Team.

3) Attach permissions (policies) to the group, while creating group that allows access to required services :-
- Amazon S3 Full Access
- AWS Lambda – Full Access
- Amazon API Gateway Administrator

4) In IAM, create a new user. Users → Add User. give username, eg dev_user1.
Select Access type :
☑ AWS Management Console Access
☑ Programmatic Access

5) During user creation, choose Add user to group. Select Dev Team.

6) Review settings → Click Create User
AWS generates credentials – Access Key IDs & Secret Key
Share login URL + password with the user.

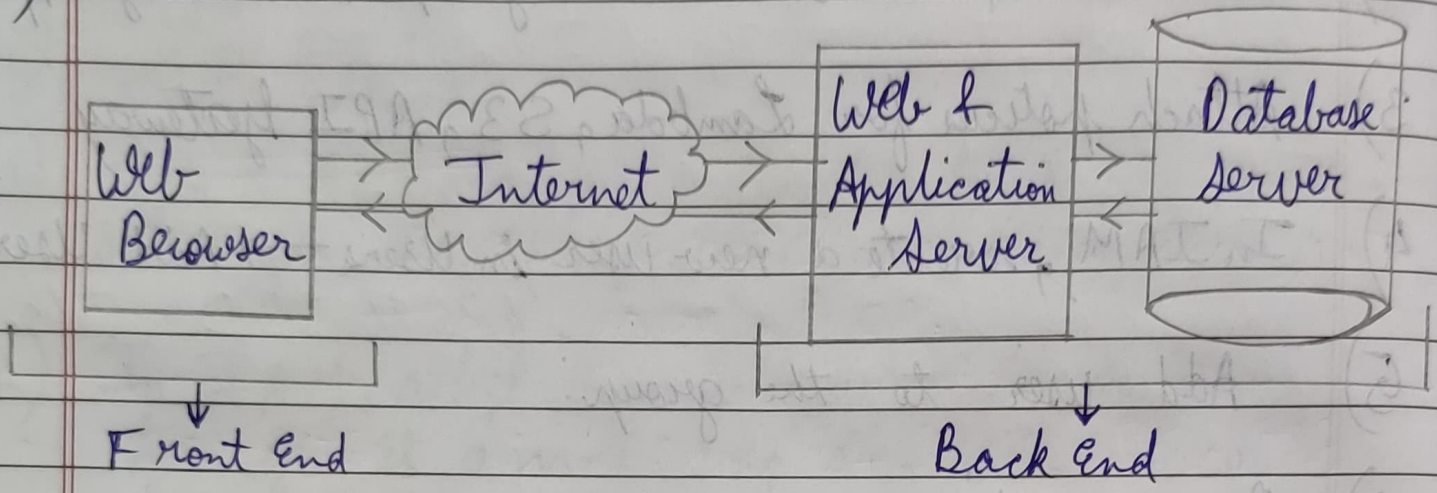
7) Log in as new user, verify access by using services.

classmate
Date
Page 11

1) Login to AWS Console. Open IAM in AWS Console.
2) In IAM, Create a User Group in User Groups
3) Attach policies for Lambda, S3, API Gateway
4) In IAM, create a new user in Users → Add User.
5) Add user to the group
6) Provide Login credentials.
7) Verify user access to services.

Q4) Cloud Computing Architecture Model diagram

Ans4)



Cloud computing Architecture is a combination of components required for a Cloud Computing Service.

It consists of several components like a frontend platform, a backend platform, servers, network, Internet Service, runtime clouds or storage spaces.

- Front End: Client uses the front End, which contains a client side interface and application. It includes web browsers, clients & mobile devices.
- Back End: It includes security mechanisms, a large amount of data storage,

- Front End — Client Side

→ Interface used by end users to access cloud services.

→ Can be a web browser, mobile app or thin client.

- Back End — Cloud Side

→ Core part of the cloud that provides computing power, storage & service management.

→ It includes servers, databases, virtual machines & storage systems.

Q5) Cloud Deployement Models

Ans 5) 4 types of cloud deployement models

1) Public Cloud

- Cloud infrastructure owned & operated by third party providers — AW, Azure & by CP.
- Cost effective, highly scalable & flexible,
- Less control, security concerns.

2) Private Cloud

- Cloud infrastructure is exclusively used by a single organization.
- Can be hosted on-premises or by a 3rd party provider.
- Provides more control, security & customization, privacy
- More expensive to maintain
- Used by Govt., Banks, large Enterprises.

3) Hybrid Cloud

- A combination of Public + Private cloud.
- Organizations keep sensitive workloads in private & use public cloud for less critical tasks.
- Flexibility & Scalibility of public cloud.
- Optimized cost of & performance
- Security & control of private cloud.

4) Community Cloud

- Cloud infrastructure shared by multiple organizations with common goals.
- Cost shared among community members
- Limited scalability
- Examples are universities, healthcare institutions or government departments.

Q6) Types of EC2 instance

Ans6) (1) General Purpose — Basic work

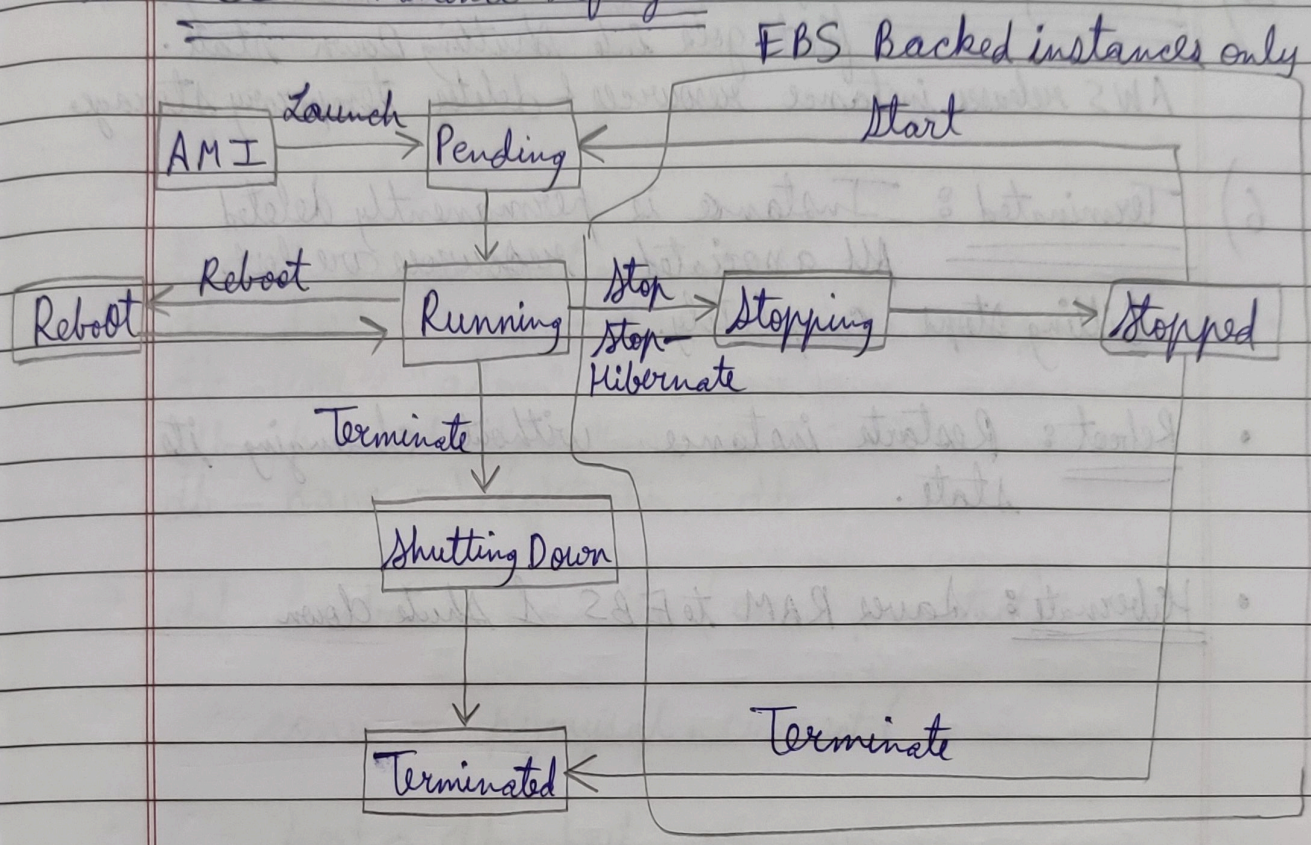
(2) Compute Optimized — High performance scientific modelling ↑ CPUs

(3) Memory optimized — ↑ memory.
in-memory database, Real time big data analytics

(4) Storage optimized — ↑ read & write
data warehousing, Hadoop distributing computed
↑ I/O

(5) Accelarated Computing — GPUs to ↑ Computation
ML, DL, 3D rendering.

* EC2 Instance Lifecycle



1) **Pending State:** When we launch a new EC2 instance, it first enters the pending state. AWS is allocating resources. You cannot access instance yet.

2) **Running State:** Once ready, instance becomes running. You can now connect, configure & use the ~~interface~~ instance. Billing starts here.

3) **Stopping:** When we issue a STOP command, instance enters into stopping. Instance performs a graceful shutdown preserving data on EBS volume.

4) **Stopped:** Instance is fully stopped, but EBS storage is retained. You can restart later without losing data. You are not charged for compute but for storage.

5) Shutting Down: When we terminate an instance, it first goes into Shutting Down state. AWS releases instance resources & deletes temporary storage.

6) Terminated: Instance is permanently deleted. All associated resources are lost. Billing stops completely.

- Reboot: Restarts instance without changing its state.
- Hibernate: Saves RAM to EBS & shuts down

* Lambda code to fetch registration details from RDS to Mysql db

→

```
import pymysql

#RDS DB connection details
db_host = "your-db-endpoint.amazonaws.com"
db_user = "admin"
db_password = "pass"
db_name = "registration_db"

def lambda_handler(event, content):

    conn = pymysql.connect(

    host= db_host,
    user = db_user,
    password = db_password,
    database = db_name)

    cursor = conn.cursor()
    cursor.execute("SELECT * FROM registrations;")
    results= cursor.fetchall()
    conn.close()

    #format output

    data =[{"id":r[0], "name": r[1], "mail":r[2]}
           for r in results]

    return {"statusCode":200, "body": data}
```

* Lambda code to delete registration details from RDS MySQL DB.

→

```
import pymysql

db_host = "your-db-endpoint.amazonaws.com"
db_user = "admin"
db_pass = "yourpassword"
db_name = "registration_db"

def lambda_handler(event, context):

    #get registration ID to delete
    reg_id = event.get("id")

    conn = pymysql.connect(

        host = db_host,
        user = db_user,
        password = db_pass,
        database = db_name)
```

* Lambda code to delete registration details from RDS MySql DB.

→

```
import pymysql

def lambda_handler(event, context):

    conn = pymysql.connect(

        host = "your-db-endpoint.amazonaws.com",
        user = "admin",
        password = "yourpassword",
        database = "registration_db")

    cursor = conn.cursor()

    #Get ID from event to delete.

    reg_id = event.get("id")

    # Delete record

    cursor.execute("DELETE FROM registrations WHERE
      id = %s;", (reg_id,))

    conn.commit()
    conn.close()

    return {"statuscode": 200, "body": f"Registration
        with ID {reg_id} deleted succesfully."}
```

* Lambda code to insert details.

→

```
import pymysql

def lambda_handler(event, context):

    conn = pymysql.connect(

        host = "your-db-endpoint.amazonaws.com",
        user = "admin",
        password = "yourpassword",
        database = "registration_db")

    cursor = conn.cursor()

    name = event.get("name")
    email = event.get("email")

    cursor.execute("INSERT INTO registrations
    (name, email) VALUES (%s, %s),",
            (name, email))

    conn.commit()
    conn.close()

    return {"statuscode": 200,
            "body": f"Registration added successfully
                for {name} ({email})."

            }
```

* Lambda code to validate login details

→ import pymysql

```
def lambda_handler(event, content):
    conn = pymysql.connect(
        host="your-db-endpoint.amazonaws.com",
        user="admin",
        password="yourpassword",
        database="registration_db")

    cursor = conn.cursor()

    email = event.get("email")
    password = event.get("password")

    cursor.execute("SELECT * FROM registrations
    WHERE email = %s AND password = %s;",
    (email, password))

    result = cursor.fetchone()
    conn.close()

    result: return {"statuscode": 200,
            "body": f"Login successful for {email}"}
```

* S3 bucket & object structure

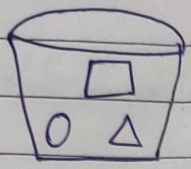
- S3 is a global service of AWS cloud based object storage service.
- Paid but very cheap service
- Used to store static content
- Object size : 0 byte to 5 TB max per object
- Buckets are region specific. Objects are stored in buckets & bucket cannot be nested.
- S3 URL

https://S3-eu-west-1.amazonaws.com/acloudguru

S3-eu-west-1 → region name

acloudguru → Bucket name

- Buckets are containers for object stored in Amazon S3, you can store any number of objects in a bucket.
- An object has data, key & metadata
- When data is added to a bucket, Amazon S3 creates a unique version ID & allocates it to the object.



- Object :- folder/ Penguins.jpg
  - folder → Key (name)
  - Penguins.jpg → Version ID
- Bucket — simplilearn
- Link — https://s3-eu-west-1.amazonaws.com/simplilearn/folder/Penguins.jpg
- By default all objects in a bucket are private

* ACL - Access control list. — S3

- It is a list of permissions attached to an object that defines who can access it & what actions they can perform
- Used to control access to resources like file, folder or AWS S3 buckets by specifying read, write or full control permission for users/groups.
- S3 bucket ACLs can grant access to

1) Individual AWS accounts
2) Public users (everyone)
3) Specific permission like READ, WRITE, FULL_CONTROL.

* S3 Object URL naming structure

https://my-bucket.S3.us-east-1.amazonaws.com/images/photo1.jpg

- my-bucket → bucket name
- S3.us-east-1.amazonaws.com → Region endpoint
- images/photo.jpg → Object key (path + file name)

* Program using Boto3 to list all S3 buckets in an AWS account

→

```
import boto3

S3 = boto3.client('S3') # create S3 client
response = s3.list_buckets() # list all buckets
print("S3 buckets in your AWS account:")
for bucket in response['Buckets']:
    print(f"- {bucket['Name']}")
```

Q) AWS Lambda? How it interacts with other cloud services.

Ans) AWS Lambda is a serverless compute service that lets us run code without provisioning or managing servers.

We simply upload our code & Lambd automatically runs it in response to events.

- We write Lambda fns, AWS automatically handles scaling, monitoring & infrastructure. We pay only for execution time.
- Interacting with other AWS services

→ S3 — Trigger a lambda fn when a file is uploaded.

→ RDS — Trigger on data insert/update events.

→ API Gateway — Expose Lambda as RESTful APIs

Q) Steps to create & configure a Lambda fn — 6M

Ans) ① Login to AWS Management Console — Go to AWS lambda service.

② Create a "New fn" by clicking on "Create fn" choose "Author from scratch".

③ Configure Basic settings

- Enter fn name
- Choose Runtime (Python, Node.js)
- Select/create an Execution Role (IAM Role with permission)

④ Write code in inline editor or upload a .zip file or connect from S3.

⑤ Configure triggers — Add event sources like S3, API gateway, CloudWatch

⑥ Save & Test the fn

- Click deploy, then Test using sample input
- View output & logs in CloudWatch.

flow { Create → Configure → Code → Trigger → deploy
↓
Test

Q) EC2 in AWS — 3M

Ans) EC2 → Elastic Cloud Compute in Amazon Web Services (AWS)

- It is a web service that provides resizable virtual servers in the cloud to run applications.
- It allows users to launch, configure & manage virtual machines i.e. instances.
- Provides scalability, flexibility & pay-as-you-use pricing.
- Provide virtual servers on demand.

Q) S3 in AWS — 3M

Ans) • Simple Storage Service — S3

- It is a scalable, secure & durable object storage service for storing & retrieving data in the cloud.
- Stores any type of data documents, images, videos
- Highly durable — 11 nines 99.999999999% & scalable
- Supports access control, versioning & lifecycle management.

*. Boto3 is the tool used to automate AWS services with Python.

Lambda code to validate login details from RDS MySQL DB

→
```
import pymysql

def lamda_handler(event, content):
    conn= pymysql.connect(
        host = "your-db-endpoint.amazonaws.com",
        user ="admin",
        password ="yourpassword",
        database ="registration-db")
    cursor = conn.cursor()
    mail = event.get("mail")
    cursor.execute("SELECT *FROM
    registrations WHERE mail = %S", (mail,))
    cursor.fetchone()
    conn.close()
    return {"statuscode":200,
            "body": f"Login successful for {mail}"
            }
```

~~8~~ 7 Lambda code to insert registration details from RDS MySql DB

→ import pymysql

```
def lambda_handler(event, context):
    conn = pymysql.connect(
        host="your-db-endpoint.amazonaws.com",
        user="admin",
        password="yourpassword",
        database="registration_db")
    cursor = conn.cursor()
    mail = event.get("mail")
    cursor.execute("INSERT INTO
    registrations (mail) VALUES (%s);",
    (mail,))
    conn.commit()
    conn.close()
    return {"statuscode": 200,
            "body": f"Registration added for
                {mail}"
    }
```

॥ ॐ नमो भगवते वासुदेवाय नमः ॥

3 November Monday

Q1) Define Cloud Computing. Types. — 3M

Ans1) Cloud Computing is the technology that enables on-demand access to shared computing resources over the internet with pay-as-you-go pricing.

Instead of owning & maintaining physical infrastructure it allows users to store, process & analyze data remotely over the cloud

Types of Cloud

- Public Cloud
- Private Cloud
- Hybrid Cloud
- Community Cloud

Q2) List two components of Cloud Architecture — 3M

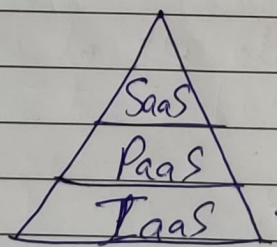
Ans 2)

Q1)(a) Cloud tech Imp in AI — 3M

Ans 1)(a)
- Cloud provides on-demand high performance CPUs/GPUs that help AI models train & run faster without needing expensive hardware.
- AI needs huge datasets, cloud provide flexible, low-cost storage to collect, manage & process this data easily.
- Cloud platforms allow quick deployment of AI models as APIs or services, making it simple for applications to use AI features anywhere.

Q1)(b) Cloud service model diagram — 4M

Ans 1)(b)



End users
OS & Application developer platform
Infrastructure & network architect

1) Software as a Service

- Provides ready to use software delivered over internet
- Used by end user. Just login & use the app — no coding no setup
- Example Gmail, Google Drive, MS office 365

2) Platform as a Service (PaaS)

- Provides a development environment like runtime frameworks.
- Used by developers building & deploying apps.
- Heroku, Google App Engine, AWS elastic Beanstalk.
- You focus on coding ~~app~~ platform handles infra & runtime.

3) Infrastructure as a Service (IaaS)

- Provides virtualized hardware resources like servers, storage & networking.
- Used by system admins & IT teams.
- You rent infrastructure instead of buying.
- AWS EC2, Google Compute Engine, Microsoft Azure VM.

Q1) (6) Steps to create user in a user group.

Ans1) (6) 1) Login to AWS Managment Console using admin credentials.

2) Open IAM – Identity & Access Managment from services menu

(3) Create a New group & name it like "Devlroup"

(4) Attach Required Policies to the group like AWS Lambda Full Access, Amazon RDS Full Access, Amazon S3 Full Access, etc.

(5) Create a new IAM User by selecting "Add User" & enabling programmatic / console access

(6) Create a New IAM User by selecting "Add User" & enabling

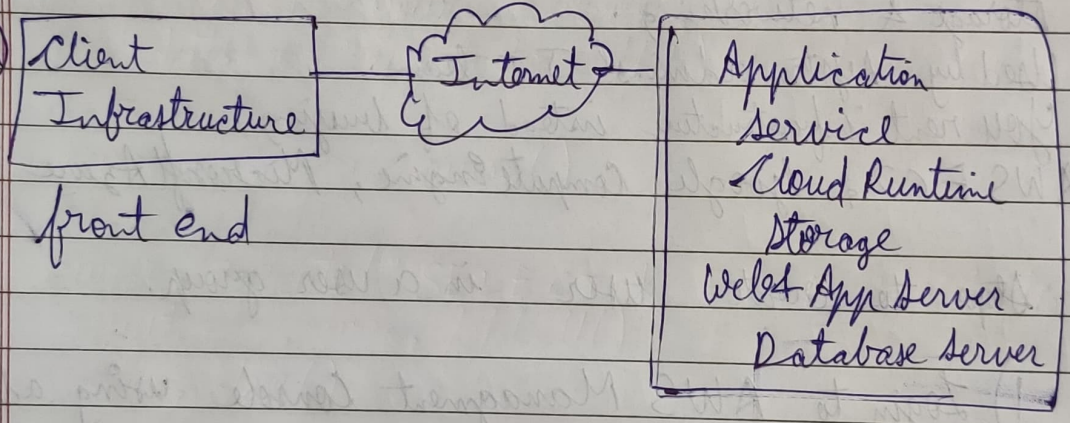
(6) Add User to the created group so the user inherits all service permissions.

(7) Review & create the user, confirming group membership & permissions.

(8) Download credentials (Access key, Secret Key, console login link) & share securely with the user.

Q2)(a) Cloud Architecture — 3 M

Ans2)(a)



1) Front end refers to the client side of cloud computing architecture which contains all the user interfaces & applications which are used by client to access cloud services like web browser

2) Backend refers to the cloud itself which is used by the service provider. It contains resources, manages resources & provides security mechanisms.

3) Application refers to the software / platform to which client accesses. It provides service in backend as per client requirement.

4) Servid in backend refers to the major three types of cloud based services like SaaS, PaaS, IaaS

5) Runtime Cloud in backend provides execution & runtime platform/environment to the VM

6) Cloud infrastructure in backend refers to the hardware & software components of cloud like servers, storage, network devices, virtualization software, etc.

2b 2b 3b 3b

Q2)(b) Fetch, delete, insert & validate — 4M

delete

Ans2)(b)

```
import pymysql
def lambda_handler(event, content):

conn = pymysql.connect(

    host="your-db-endpoint.amazonaws.com",
    user="admin",
    password="yourpassword",
    database="registration-db")

cursor = conn.cursor()
email = event.get("email") # input value to delete

cursor.execute("DELETE FROM registrations WHERE
    email =%s", (email,))
conn.commit()
conn.close()
```

Q2)(b) Fetch registration details. — 4M

Ans2)(b)

```
import pymysql

def lambda_handler(event, content):

    conn = pymysql.connect(
        host = "your-db-endpoint.amazonaws.com",
        user = "admin",
        password = "your_password",
        database = "registration_db")

    cursor = conn.cursor()

    cursor.execute("SELECT * FROM registrations")
    result = curso.fetchall()
    conn.close()
```

Q3)(b) Insert details. — 4M

Ans3)(b)

```
import pymysql

def lambda_handler(event, content):

    conn = pymysql.connect(
        host = "your-db-endpoint.amazonaws.com",
        user = "admin",
        password = "your_password",
        database = "registrations_db")
```

```
cursor = conn.cursor()

name = event.get("name")

cursor.execute("INSERT INTO registrations
(name) VALUES (%s,)", (name,))

conn.commit()
conn.close()
```

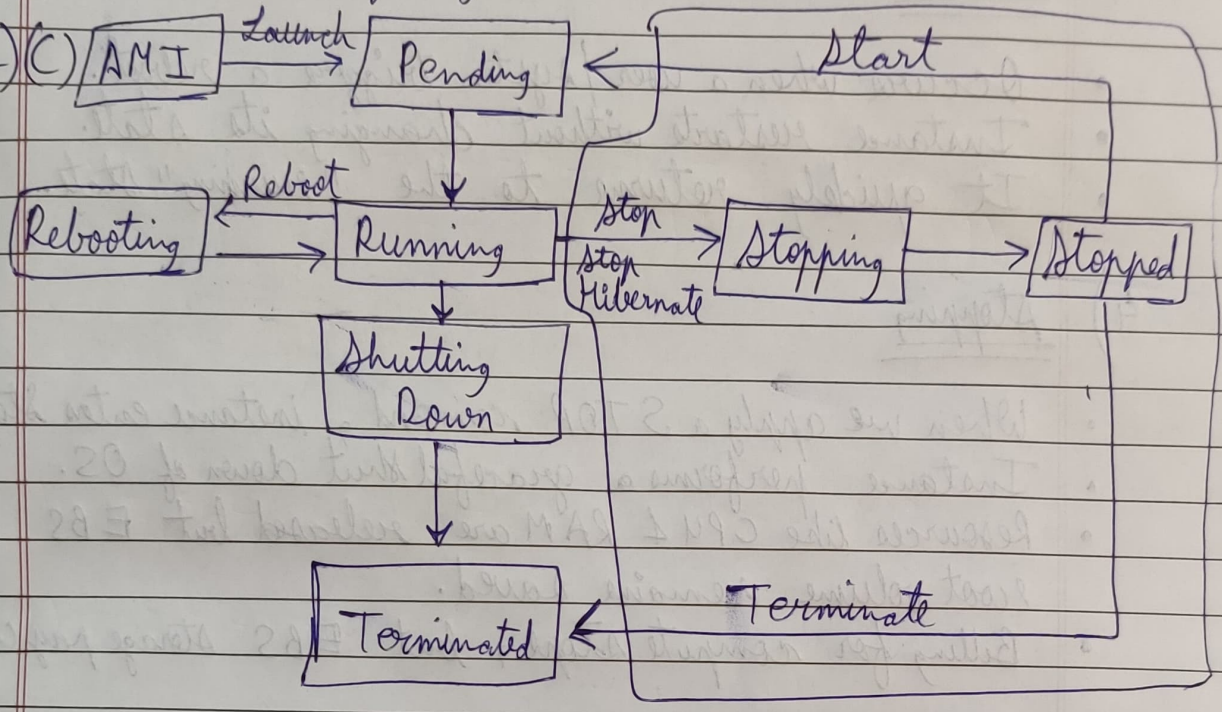
Q3)(b) validate login details ——— 4M

Ans3)(b) import pymysql

```
def lambda_handler(event, content):
conn = pymysql.connect(

    host="your-db-endpoint.amazonaws.com",
    user="admin",
    password="yourpassword",
    database="registration-db")

cursor = conn.cursor()

name = event.get("name")
password = event.get("password")
cursor.execute("SELECT * FROM users WHERE
name=%s AND password=%s", (name,
password))

result = cursor.fetchone()
conn.close().
```

Q2)(C) Instance life cycle — 8M

Ans 2)(C)



EBS Backed instance only.

1) Pending

- This state starts when you launch an EC2 instance
- AWS is allocating hardware, networking & preparing VM
- We are not billed for compute until it reaches running state.
- We cannot access instance yet.

2) Running

- The instance is fully started, OS is active.
- We can connect, configure & use the instance.
- Billing for compute begins in this state.

3) Rebooting

- Occurs when a user/system triggers a reboot.
- Instance restarts without changing its state.
- It quickly returns to the "running" state.

4) Stopping

- When we apply a STOP command, instance enters Stopping
- Instance performs a graceful shut down of OS.
- Resources like CPU & RAM are released but EBS root volume remains saved.
- Billing for compute stops, but EBS storage pay continues

5) Stopped

- Instance is fully stopped but EBS storage is retained.
- You can start later without losing data.
- You are not charged for compute but for storage.

6) Shutting-Down

- This state appears after Terminate command is applied.
- AWS is in the process of deleting instance resources

7) Terminated

- Instance is permanently deleted & cannot be restarted.
- All associated resources are lost.
- Billing stops completely.

Q3)(a) ACL in S3 — 3M

Ans3)(a) Access Control Lists

- A permission mechanism used to control access to individual S3 buckets & objects.
- It defines which AWS accounts or users can read, write or manage a specific bucket or object.
- When a request is recieved against a resource, Amazon S3 checks the corresponding ACL to verify that the requester has the necessary access permissions or not.

Q3)(C) Steps to create & configure a lambda fn — 8M

Ans3)(C)